



AIPATHS ACADEMY

PRACTICAL ROADMAP

Your first AI agent system

7 steps to turn scattered tasks into a useful system for your business.

AUTHOR

Gonzalo Sabaino

WEB

AIPaths Academy

Before you start

If you use AI today to write, think, summarize, research or code, but you still feel like everything depends on you, this guide is for you.

The problem is not that AI does not work.

The problem is that we often use it as a standalone tool: you open a chat, explain the context, ask for something, fix the result and start over again the next day.

That helps.

But it does not scale.

This guide is not a list of tools or a collection of prompts.

It is a map to decide which part of your business should become your first AI agent system.

By the end, you should be able to answer three questions:

1. Which process in my business should I improve first with agents?
2. What type of system do I need for that use case?
3. What is the first concrete step to start without improvising?

The goal is not to learn more names.

The goal is to stop trying AI at random and start designing a more repeatable, clearer and more useful way of working for your business.

The problem is not AI. It is that every task starts over.

Most people who say "I use AI for work" are really doing something like this:

- they open ChatGPT,
- paste some context,
- ask for a task,
- fix the result,
- start from zero again the next day.

That helps a lot.

But it has a limit: every time you want to move forward, the process depends on you again.

You have to remember what to ask, what context to copy, which examples to show, what result you expect, what to correct and what to do next.

AI can write, summarize, analyze or suggest ideas.

But you are still the one holding the continuity together.

That is why many people feel something strange: they use AI every day, but their business does not feel more organized, more automated or easier to run.

It is not because they are using AI wrong.

It is because they are still using it as isolated conversations, not as part of a work system.

The important leap is not going from a bad prompt to a perfect prompt.

The leap is to stop starting from zero every time.

The goal of this guide is to help you choose a repeated task in your business and turn it into your first simple AI agent system.

The 3 levels of AI maturity

Before choosing a tool, identify where you are.

Level	What you use	What it is useful for	Main limit
1. Chat and copilots	ChatGPT, Claude, Gemini	Think, write, summarize, analyze and solve one-off tasks	Everything depends on you carrying the context
2. Workflows and automations	n8n, Make, Zapier	Connect apps and repeat clear processes	They struggle to decide when context changes
3. Agent systems	OpenClaw, agent operating systems, multi-agent environments	Give AI roles, memory, tools and continuity	Requires good system design

Most people are somewhere between level 1 and level 2.

They use AI every day, but their business does not feel much more automated.

That feeling is normal.

Because using AI is not the same as having an AI system.

Roadmap 7 steps

Build your first agent system

This roadmap is not a universal recipe or a list of tools.

It is a way to decide which process in your business should become an agent system first: which task to choose, what type of system to design, what context to give it and how to know if it actually works.

Step 1: Choose a concrete task

Before talking about agents, tools or automation, you need to choose a real task.

Do not start with: "I want to use agents."

Better question: "Which part of my work repeats often and would work better if it were more organized?"

A good first task does not need to be big. It needs to be clear.

Choose a task that meets at least 3 conditions:

- ☐ it happens every week,
- ☐ it takes real time,
- ☐ it has a clear result,
- ☐ you can review whether it worked or not,
- ☐ nothing serious breaks if the first version gets it wrong.

For example:

- review metrics and detect what changed,
- turn loose ideas into content drafts,
- research topics or competitors,
- organize comments and frequent questions,
- prioritize pending tasks,
- prepare replies for customers or community.

This matters because your first system should not try to automate your whole company.

If the task sounds like "manage marketing", it is still too big. If it sounds like "suggest 5 content ideas every week based on these inputs", it can already be designed.

A concrete task gives you boundaries: what it receives, what it produces, how it is reviewed and when it should ask for help.

"I want it to deliver a metrics summary every Monday."

"I want it to propose 5 prioritized content ideas each week."

"I want it to review frequent questions and tell me which answers we should create."

That can already be designed.

Result of this step: you have chosen a concrete, frequent and low-risk task.

Step 2: Turn the task into a small system

A task is what you do today. A system is the ordered way to make that task happen with less improvisation.

Too broad: "I need a marketing agent."

Better defined: "I need a system that detects topics, prioritizes them and gives me 3 content angles every week."

When you formulate it as a system, the important decisions become visible:

- **Input:** what information it receives.
- **Process:** what it needs to analyze, organize or decide.
- **Output:** what useful result it should deliver.
- **Cadence:** when or how often it happens.
- **Review:** who validates whether the result is useful.

This exercise also shows whether you chose the right task. If you cannot name the input, the output or who reviews the result, the scope is still too open.

In that case, shrink the scope until you can describe the flow in one sentence. A good first system receives something known, applies a simple criterion and delivers an output you can use, correct or discard.

The first version can be simple: one agent, good context and a clear output. If it works, you can later add memory, tools, recurring tasks or more specialized agents.

The key: an agent is a role; a system is the full process that turns an input into a useful result.

Result of this step: you know which system you are going to build and the first output it should produce.

Some possible first systems:

If you want to improve...	Recommended first system	First useful output
Publishing more content	Content system	Prioritized ideas, angles and drafts
Understanding market or competitors	Research system	Summary of signals, competitors and opportunities
Measuring what is working	Reporting system	Key metrics, changes and recommendations
Replying to community or support	Support/community system	FAQs and suggested replies
Organizing tasks and follow-up	Operations system	Priorities, blockers and next steps
Getting or qualifying leads	Sales/leads system	Prioritized list and next messages
Coding faster	Coding/prototyping system	Small change, reviewed and tested
Organizing internal knowledge	Documentation system	Summaries, SOPs and internal FAQ base
Improving onboarding	Onboarding system	Checklist, materials and next steps by profile
Understanding user feedback	Feedback/product system	Recurring themes, friction and opportunities
Tracking simple finances	Basic finance system	Alerts, notable expenses and monthly summary

Step 3: Choose the tool category

Not long ago, using AI almost always meant opening a chat and asking for something. Today the map is broader: chat, code, app automation, technical agents and multi-agent systems.

That is why it is easy to get confused. Not all AI tools compete with each other. The question is not "what is the best tool", but "what type of work do I want to improve".

Chats and copilots

The simplest way to use AI: you explain what you need and receive an answer. They are useful for thinking, writing, summarizing, analyzing documents, generating ideas or solving one-off tasks. Their limit is that each conversation depends on you bringing the context and pushing the work forward.

Examples: ChatGPT, Claude, Gemini, Perplexity.

AI-assisted apps and IDEs

An AI-assisted IDE helps you write, understand and modify code inside a visual interface. It is ideal when the work happens in a software project: an app, a website or a digital product. Its limit is that it works inside code, not across full marketing, content, support or operations processes.

Examples: Cursor, Windsurf, GitHub Copilot, editors with built-in AI.

Terminal agents or CLI agents

These are technical agents used from the terminal. They can read files, edit repositories, run commands, inspect errors and execute tests. They are useful when you want AI to work on a real software project and leave changes ready to review. Their limit is that they require comfort with code and the terminal.

Examples: Claude Code, Codex CLI, Gemini CLI, Aider, OpenCode.

Visual automation

These tools connect applications and create flows where an event in one app triggers an action in another. They are ideal for clear, repeatable processes with stable rules. Their limit appears when the process needs judgment, interpretation, memory or context-aware decisions; then they can become fragile.

Examples: n8n, Make, Zapier.

Agent operating systems

They are useful when you want something closer to a team of agents than a standalone tool. They let you create agents with roles, context, tools, channels, scheduled tasks and continuity. They are ideal when you need memory, recurring work or several agents working on the same business. Their limit is that they require design: if the process is unclear, a multi-agent system will not fix it.

Examples: OpenClaw, agentic OS-style systems, multi-agent environments.

Frameworks for building custom agents

A framework is a technical foundation for building agents inside your own application, product or infrastructure. It is useful when you need deep control over how the system reasons, acts and integrates. It is ideal for technical teams, but usually not the best entry point for improving operations now.

Examples: LangGraph, AutoGen, CrewAI, custom frameworks.

A simple rule to decide

- if it is a one-off task, start with chat or a copilot;
- if it happens inside code and you want a visual interface, use an AI-assisted IDE;
- if it happens inside code and you want more technical control, use a terminal agent;
- if it connects apps with clear rules, use visual automation;
- if it requires role, memory, tools and continuity, use an agent system;
- if you are building your own agent-powered product, use a framework.

Result of this step: you know whether your use case needs chat, an AI-assisted IDE, a terminal agent, visual automation, an agent operating system or a framework.

Step 4: Design the context

Context turns generic AI into a useful assistant for your business. Without context, the agent improvises; with good context, it starts working like someone who understands how you operate.

Think of it like onboarding a new employee: you would not expect good decisions without explaining what you sell, who you help, what tone you use, what good examples look like, what it should not promise and when it needs approval. The same applies to an agent: context prevents you from explaining everything every time and gives it criteria to act as a specialized assistant.

Your agent needs context about:

- **the business:** what you sell, who you help, what your offer is;
- **the audience:** who your customer is, what problems they have, what they care about;
- **the task:** what it needs to do, how often and for what objective;
- **the criteria:** how to decide whether something is good, important or relevant;
- **the examples:** which outputs are good and which are not;
- **the sources:** what information it can use and what it should ignore;
- **the limits:** what it must not do and when it should ask for approval.

For example: a content agent needs to understand what you sell, who you speak to and what kind of ideas are good for your brand. A reporting agent needs to know which metrics matter. A support agent needs to know your policies, prices, promises and limits.

You do not need to write an infinite document. The point is to give it the right context: enough to stop guessing, specific to the task and clear about which sources it can use, what it must not do and when it needs human approval.

Result of this step: you are clear on what the agent needs to know, what criteria it should use and which limits it must respect.

Step 5: Define inputs, outputs and success criteria

You already have a task, a system and the context the agent needs. Now you need to give it boundaries: what goes in, what comes out and how it will be evaluated. An agent without clear boundaries can work a lot and still not help; if it does not help you make a decision, save time or move a task forward, the system is not working.

Input is what the agent receives to work with: data, documents, ideas, comments, metrics, links, tasks or instructions.

Output is what it must deliver: a summary, a recommendation, a draft, a prioritized list, a report, a suggested reply or a concrete action.

Success criterion is how you will know whether that was useful.

Example: weekly YouTube reporting system.

- **Receives:** channel metrics, published videos, relevant comments and web traffic from YouTube.
- **Delivers:** executive summary, top videos, anomalies and a recommendation for next week.
- **Works if:** it helps decide what to publish, what to improve or what to stop doing.

It is also useful to define when it should run and where the result should appear, based on the apps, channels or systems you already use for work.

The first version can be manual: you provide the input, review whether the output helps and only then connect it to data, tools or recurring tasks.

Result of this step: you define what the system receives, what it produces, where it delivers it and how you will know if it works.

Step 6: Add memory, tools and automation

This step comes after validating the simple version, not before. First you need to know that the chosen task makes sense, that the context helps and that the output is actually useful.

Memory, tools and automation do not fix a confusing system: they amplify it. Add them when you already know which result you want to repeat, improve or make more continuous.

What to add and when:

Memory: when the system needs to remember decisions, preferences, good examples, learnings, mistakes to avoid or business rules.

Tools: when it needs to read data, check files, use a browser, connect to an API, review a calendar, publish to a channel or execute commands.

Automation: when the output is already useful and should happen continuously: scheduled reports, alerts, reviews, recurring drafts or handoffs to another agent.

The order matters: if you add memory too early, you save noise; if you connect tools without a clear output, you add complexity; if you automate a poorly defined task, you repeat the mess faster.

1. First: an agent with good context that delivers a useful output when you ask.
2. Then: turn it into a weekly or recurring task.
3. Next: connect it to data or tools.
4. Finally: make it notify, compare, remember and coordinate with other agents.

Result of this step: you decide what the first version needs and what should only be added after the system has proven useful.

Step 7: Measure cost, reliability and improvement

An agent system is not evaluated by how intelligent it seems. It is evaluated by whether it improves a real operation: whether it saves time, improves decisions or reduces repeated work.

At the beginning you do not need to measure ten things. Choose 2 or 3 simple signals and review them every week.

- Does this save me real time?
- Does it deliver something I actually use?
- Does it reduce repeated work?
- Does it help me make better decisions?
- Does it need too much human correction?
- Do I know how much it costs to maintain?

It is also useful to review errors, because not all of them mean the same thing:

Format: adjust the output.

Context: add missing information, examples or limits.

Criteria: clarify the decision rules.

Tool: limit the integration or add human approval.

Improvement usually comes from reviewing those errors and adjusting context, examples, output, limits or approvals. The goal is not full autonomy; it is reliable leverage: the system helps you in a repeatable way without demanding more energy than it saves.

Result of this step: you know whether the system is worth it, what you should improve and what should not be automated yet.

Complete example: competitor analysis

Suppose you have a B2B company, an agency, a consultancy or a digital product. You want to understand what your competitors are doing, but today the process is irregular: you check websites when you remember, review scattered social posts and end up with loose links.

The problem is not getting information. The problem is turning that information into decisions.

Step 1: Choose a concrete task

What I choose: review a defined group of competitors every week and detect relevant changes, opportunities and risks.

Why: "doing market research" is too broad. Reviewing defined competitors with a clear cadence lets you answer: what changed, what matters and what decision could come from it.

Step 2: Turn the task into a small system

What I choose: a weekly competitor analysis system.

Why: you do not only need an agent that searches things on the internet. You need a process that turns scattered signals into a useful report for decisions.

- monitor defined competitors,
- review changes in websites, offers, content or positioning,
- summarize relevant signals,
- separate noise from useful information,
- detect opportunities and recommend next steps.

Step 3: Choose the tool category

What I choose: an agent operating system.

Why: this case needs continuity, web tools, memory of changes and recurring reports. You can validate it first with a chat and a clear competitor list, but if you want it to work every week, the natural category is an agent system.

Step 4: Design the context

What I give the agent: what you sell, who you help, what your main offer is, who your competitors are, which market you want to watch, which signals matter and which decisions you want to make from the analysis.

Why: monitoring competitors to adjust pricing is not the same as monitoring them to improve content, detect product opportunities or find sales angles. Context tells the agent what to watch and what to ignore.

Step 5: Define inputs, outputs and success criteria

Inputs: competitor list, websites, landing pages, newsletters, public posts, social profiles, visible ads, prices, offers and internal notes about your positioning.

Weekly output: summary of important changes, competitors that moved the most, new offers or messages detected, opportunities to differentiate, risks to review and 3 actionable recommendations.

Success criteria: it saves manual research, detects moves that used to go unnoticed, separates signal from noise and helps make decisions about offer, content, positioning or sales.

Step 6: Add memory, tools and automation

What I add after validating: memory to compare weeks, tools to check sources and automation so the report appears without starting from zero.

Memory: previous changes, historical messages, competitors that usually move more, signals that mattered before and things already reviewed.

Tools: browser or web tools, internal documents, competitor database, relevant public sources and the channel where the report should be delivered.

Automation: weekly report, alerts for important changes, tracking of new content or campaigns and monthly pattern summary.

Step 7: Measure whether it is worth it

What I look at: whether the system helps you see the market more clearly each week. It does not need to be perfect; it needs to help you make better decisions without requiring more correction than it saves.

- you find changes you were not seeing before,
- the report saves manual review,
- the recommendations are actionable,
- it improves your positioning, content, offer or sales.

Closing

This roadmap gives you the map: choose a concrete task, turn it into a system, give it context, define inputs and outputs, add memory and tools when it makes sense, and measure whether it really improves your operation.

The next step is to build a first version that works.

At AIPaths we use OpenClaw because it gives us a flexible foundation to design agent systems without being tied to a single model. We can combine local models for private information with OpenAI or Anthropic models when we need more capability, while keeping context, memory, tools and continuity inside the same system.

That is the important part: you do not have to rebuild everything every time the model, tool or provider changes.

If you want to go from this roadmap to a real implementation, we put together practical training to build your first system with OpenClaw.

The goal is that you finish with:

- OpenClaw installed,
- your first agent configured,
- context designed for your business,
- a channel to talk to your agent,
- a recurring task and a first useful report,
- a structure you can adapt to other processes.

To go from roadmap to implementation:

[View the OpenClaw training](#)

[Community](#) | [Contact](#) | [X / Twitter](#) | [GitHub](#)